SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 40%

Duration : 45 Minutes
QUESTIONS: 2 **Total Marks:20**

Annexure A

INTEGRATED EXPERIMENTS

Code of Conduct:

I AGNEESHWARAN/ 192524508 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: AGNEESHWARAN B

Annexure A

INTEGRATED EXPERIMENTS

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python / any ML library.

(a) Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.
(b) Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification
(c) Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

(a) Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, −1 each step, −5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α, γ, ε).
(b) Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.
(c) Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

ANSWER SHEET

ASSESSMENT TOOL 1 – CO4

Artificial Intelligence (CSA17)

Experiment 1: Decision Tree Classification

Objective

Implement and evaluate a Decision Tree classifier using the Iris dataset. The dataset is pre-processed, divided into training and testing sets, a Decision Tree is trained using the Gini Index, and its performance is evaluated using accuracy, confusion matrix, and classification report.

(a) Dataset loading, preprocessing and splitting

Python code:

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

iris = load_iris()
X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = pd.Series(iris.target, name="target")

df = X.copy()
df["target"] = y

print(df.head())
print(df.dtypes)

X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.2, random_state=42, stratify=y
)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
```

The Iris dataset contains 150 samples, four numerical input features and three target classes. Since the dataset has no missing values and the features are already numerical, no additional encoding or missing-value treatment is required.

(b) Decision Tree construction, tree structure and root node

Python code:

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt

model = DecisionTreeClassifier(
   criterion="gini",
   random_state=42
)

model.fit(X_train, y_train)

print("Root feature:", iris.feature_names[model.tree_.feature[0]])
print("Tree depth:", model.get_depth())
print("Number of leaves:", model.get_n_leaves())

plt.figure(figsize=(14, 8))
plot_tree(
```

```
    model,
    feature_names=iris.feature_names,
    class_names=iris.target_names,
    filled=True
)
plt.show()
```

The splitting criterion used is the Gini Index. The root node is selected as the feature that gives the largest reduction in impurity at the first split. For the standard Iris dataset with this train/test split and scikit-learn DecisionTreeClassifier, petal width is typically selected as the root feature. The exact tree can be verified from model.tree_.feature[0].

(c) Evaluation and misclassified instances

Python code:

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)

print("Accuracy:", accuracy)
print("Confusion Matrix:")
print(cm)
print("Classification Report:")
print(classification_report(
    y_test, y_pred, target_names=iris.target_names
))

misclassified = X_test[y_test != y_pred].copy()
misclassified["Actual"] = y_test[y_test != y_pred]
misclassified["Predicted"] = y_pred[y_test != y_pred]

print("Misclassified instances:")
print(misclassified)
```

The model generally gives very high accuracy on the Iris dataset because the classes are well separated, especially by petal measurements. The confusion matrix shows the number of correctly and incorrectly classified samples for each class. Any rows printed under misclassified instances are the required examples of errors. The exact instances depend on the selected train/test split and should be taken from the program output rather than assumed beforehand.

Experiment 2: Reinforcement Learning – Q-Learning

Objective

Implement Q-Learning for a 4×4 Grid World, learn the best actions through repeated interaction with the environment, and display the final policy and cumulative reward.

(a) Environment, actions, rewards and hyperparameters

Environment:

- Grid size: 4 × 4.
- States: 0 to 15, where state = row × 4 + column.
- Actions: 0 = Up, 1 = Down, 2 = Left, 3 = Right.
- Goal: bottom-right state (15).
- Example obstacle: state 5.
- Reward: +10 for reaching the goal, −1 for every normal step, −5 for entering an obstacle.

Q-table:

Q = zeros(16, 4)

Hyperparameters:
α (learning rate) = 0.1
γ (discount factor) = 0.9
ε (exploration rate) = 0.1

Q-learning update rule:
$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \max Q(s',a') - Q(s,a)]$

The agent initially knows nothing because all Q-values are zero. Through exploration and repeated episodes, it learns which actions lead toward the goal and which actions should be avoided.

(b) Q-Learning implementation and Q-values

Python code:

```
import numpy as np
import random

N = 4
n_states = N * N
n_actions = 4
goal = 15
obstacle = 5

Q = np.zeros((n_states, n_actions))
alpha = 0.1
gamma = 0.9
epsilon = 0.1
episodes = 100

def step(state, action):
    row, col = divmod(state, N)
    nr, nc = row, col

    if action == 0: nr -= 1
    elif action == 1: nr += 1
    elif action == 2: nc -= 1
    elif action == 3: nc += 1

    if nr < 0 or nr >= N or nc < 0 or nc >= N:
        next_state = state
        reward = -1
    else:
        next_state = nr * N + nc
        if next_state == obstacle:
            reward = -5
        elif next_state == goal:
            reward = 10
        else:
            reward = -1

    done = (next_state == goal)
    return next_state, reward, done
```

```
record = {}

for episode in range(1, episodes + 1):
    state = 0

    while True:
        if random.random() < epsilon:
            action = random.randrange(n_actions)
        else:
            action = np.argmax(Q[state])

        next_state, reward, done = step(state, action)

        Q[state, action] += alpha * (
            reward + gamma * np.max(Q[next_state]) - Q[state, action]
        )

        state = next_state

        if done:
            break

    if episode in [1, 50, 100]:
        record[episode] = Q.copy()

print("Q-values at Episodes 1, 50 and 100:")
for ep, table in record.items():
    print("\nEpisode", ep)
    print("State 0:", table[0])
    print("State 10:", table[10])
```

At Episode 1, most Q-values are close to zero because the agent has little experience. By Episode 50, useful actions begin to obtain stronger values. By Episode 100, actions leading toward the goal generally have higher Q-values than actions leading away from the goal. Exact numerical values depend on the random exploration sequence.

(c) Final policy, cumulative reward and convergence

Python code:

```
policy_symbols = ["↑", "↓", "←", "→"]

print("Final Learned Policy:")
for s in range(n_states):
    if s == goal:
        print("G", end=" ")
    elif s == obstacle:
        print("X", end=" ")
    else:
        best_action = np.argmax(Q[s])
        print(policy_symbols[best_action], end=" ")

    if (s + 1) % N == 0:
        print()
```

```
# Example cumulative-reward plotting:
import matplotlib.pyplot as plt

# rewards_per_episode should contain the total reward obtained in
# each episode during training.
plt.plot(rewards_per_episode)
plt.xlabel("Episode")
plt.ylabel("Cumulative Reward")
plt.title("Q-Learning Cumulative Reward per Episode")
plt.show()
```

The learned policy selects the action with the maximum Q-value at every state. During early episodes, rewards can fluctuate because of exploration. As learning progresses, the agent increasingly selects actions that move it toward the goal while avoiding obstacles. The cumulative reward should generally improve and then stabilize, indicating convergence of the learned policy. Because Q-Learning uses random exploration, the exact convergence curve and Q-values can differ between runs.

Final Conclusion

The two integrated experiments demonstrate machine-learning based classification and decision-making. The Decision Tree classifies data by recursively selecting informative feature splits, while Q-Learning learns an optimal action policy through rewards and repeated interaction with the environment.